Universidade Federal de Itajubá – UNIFEI

Instituto de Engenharia de Sistemas e Tecnologias da Informação – IESTI

ECOM06 – Compiladores

Projeto de Compilador
A Linguagem HolyPy

Analisador Léxico e Sintático

Gustavo Inácio Arraes Fernandes

Itajubá, MG – 2026

Sumário

1 Introdução **2**

1.1 Contexto e Motivação 2

1.2 Inspiração: O Legado Técnico de Terry A. Davis e o HolyC 2

1.3 Arquitetura do Compilador 3

2 Metodologia **4**

2.1 Concepção do Vocabulário 4

2.2 Expressões Regulares 5

2.3 Autômatos Finitos Determinísticos (AFDs) 5

2.4 Regras de Produção 6

3 Resultados **6**

3.1 Especificações da Linguagem 6

3.2 Tabela de Tokens e Expressões Regulares 7

3.3 Regras de Produção 14

3.4 Autômatos Finitos 17

4 Conclusão **26**

4.1 Disponibilização e documentação do projeto 27

1 Introdução

1.1 Contexto e Motivação

O presente documento descreve o projeto de um compilador desenvolvido para a disciplina ECOM06 – Compiladores, da Universidade Federal de Itajubá. O resultado central desse projeto é a linguagem **HolyPy**, uma linguagem de programação híbrida de propósito geral que combina a legibilidade sintática do Python com o controle de tipos e de memória característico de C e Rust.

O objetivo central do trabalho é construir, de forma progressiva, as três fases iniciais de um compilador: o Analisador Léxico, responsável pela tokenização do código-fonte; o Analisador Sintático, que verifica a estrutura gramatical do programa; e, como extensão natural, o Analisador Semântico e um Gerador de Código C, cujo alvo final é a produção de código C compilável por `gcc` ou `clang`.

1.2 Inspiração: O Legado Técnico de Terry A. Davis e o HolyC

O nome **HolyPy** é uma referência direta ao HolyC, a linguagem de programação feita por Terry A. Davis como núcleo de seu sistema operacional, o *TempleOS*. Do ponto de vista técnico, o feito de Davis é notável: ao longo de mais de uma década de trabalho solitário, ele projetou e implementou um sistema operacional completo, incluindo *kernel*, compilador, sistema de arquivos, interface gráfica e jogos, do zero, sem colaboradores. Essa capacidade de engenharia de baixo nível, aliada à disciplina para manter um projeto de tamanho tão grande, constitui um legado técnico grandioso e objeto de estudo em comunidades de computação.

Porém, qualquer análise honesta do trabalho de Davis exige o reconhecimento de sua trajetória pessoal. Davis sofria de esquizofrenia severa, condição diagnosticada ao longo de sua vida adulta e que progressivamente o desconectou da realidade, provocando delírios, comportamentos erráticos e falas públicas de caráter preconceituoso e ofensivo. Sua história serve como lembrete sobre a importância do suporte psiquiátrico e do acolhimento social para pessoas com transtornos mentais graves. Davis faleceu em 2018, em condição de

vulnerabilidade social.

O trabalho não homenageia qualquer conduta ou discurso do autor do TempleOS. A referência é restrita ao espírito técnico do "do it yourself" e à curiosidade pelo funcionamento interno dos computadores, elementos que motivam igualmente a construção deste compilador.

1.3 Arquitetura do Compilador

O compilador **HolyPy** seguirá, próximas etapas de implementação, um *pipeline* definido pelos slides da disciplina, de fases sequenciais, conforme descrito abaixo e ilustrado na Figura 1.

1. **Lexer**: recebe o código-fonte `.hpy` e produz uma sequência de *tokens*.
2. **Parser**: aplica *parsing* descendente recursivo sobre os *tokens* e constrói a *Abstract Syntax Tree* (AST).
3. **Analisador Semântico**: percorre a AST verificando tipos, escopos e declarações.
4. **Gerador de Código** transforma a AST anotada em código C válido.
5. ***Backend* C**: o código gerado é compilado por `gcc` ou `clang` para produzir o binário final.

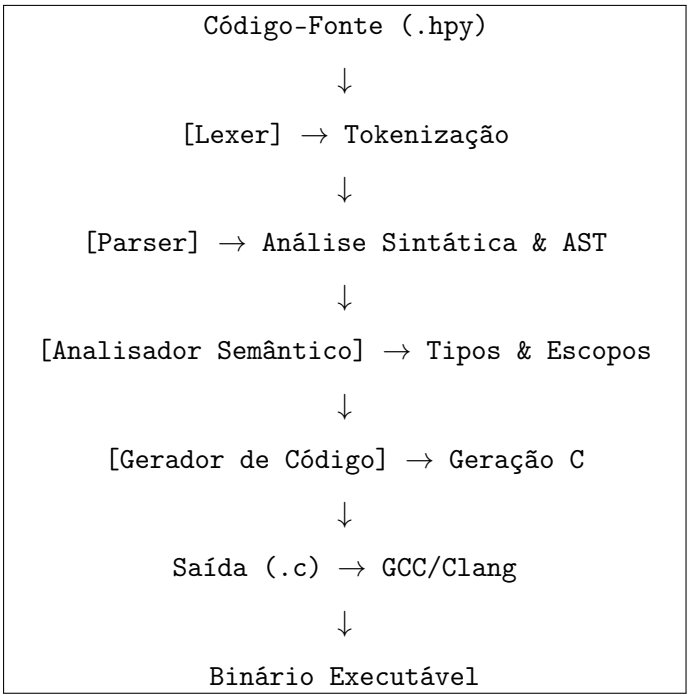


Figura 1: *Pipeline* de compilação do **HolyPy**.

O lexer e o parser vão ser implementados em Python, com alguns testes em C/C++. Modos de compilação adicionais (`-emit-tokens`, `-emit-ast`, `-emit-c`, `-run`) permitem inspecionar a saída de cada fase individualmente.

2 Metodologia

2.1 Concepção do Vocabulário

A primeira etapa consistiu na definição do vocabulário da linguagem: quais símbolos, operadores e palavras-chave seriam reconhecidos pelo Analisador Léxico. Optou-se por um conjunto pequeno, mas completo, capaz de suportar os requisitos exigidos pelo enunciado do trabalho: tipos de dados inteiro, real e caractere; comandos de atribuição, entrada, saída, condicionais e repetição; operadores relacionais, lógicos e aritméticos; e delimitadores de bloco.

Cada elemento do vocabulário foi mapeado a um *token*, identificado por um nome simbólico prefixado com `T_`. Tokens compostos (ex.: `T_INT` agrupando `i8`, `i16`, `i32` e `i64`) foram

definidos para simplificar as regras gramaticais, preservando o lexema original como atributo para uso na fase semântica.

2.2 Expressões Regulares

Com o vocabulário definido, foram construídas as expressões regulares (ERs) correspondentes a cada token. As ERs foram elaboradas observando os seguintes princípios:

- ***Maximal Munch***: operadores compostos (`->`, `>=`, `<=`, `==`, `!=`) são testados antes dos simples para evitar tokenização incorreta.
- **Prioridade entre palavras reservadas e identificadores**: palavras-chave e tipos são verificados antes de classificar um lexema como `T_ID`, evitando colisões (ex.: `cycle` não deve ser lido como identificador).
- **Atribuição *vs.* Igualdade**: o lexema `=` gera `T_ATR` e `==` gera `T_EQ`; o *maximal munch* garante a distinção.
- **Diretiva *vs.* Comentário**: `!` inicia uma diretiva de modo (ex.: `!embedded`) e `#` inicia um comentário de linha, não sendo intercambiáveis.

2.3 Autômatos Finitos Determinísticos (AFDs)

Para cada expressão regular, foi construído um Autômato Finito Determinístico equivalente utilizando o software Excalidraw para diagramação. O processo seguiu a sequência metodológica padrão da teoria de linguagens formais:

1. Definição da ER para o token.
2. Construção do AFN (Autômato Finito Não Determinístico) pelo método de Thompson, que converte subexpressões em sub-autômatos conectados por transições ε.
3. Conversão do AFN em AFD pelo método da construção de subconjuntos (*subset construction*), eliminando o não-determinismo.
4. Minimização do AFD, quando aplicável, por fusão de estados equivalentes.

Os diagramas resultantes estão apresentados na Seção 3.4 deste documento.

2.4 Regras de Produção

Após a definição léxica, foram elaboradas as regras de produção da gramática livre de contexto que descreve a sintaxe de **HolyPy**. As produções cobrem todas as construções da linguagem: declarações de variáveis e funções, estruturas de controle de fluxo, expressões aritméticas e lógicas, chamadas de função, comandos de entrada/saída e gerenciamento de memória. A estratégia de *parsing* adotada é a descida recursiva (*recursive descent*).

3 Resultados

3.1 Especificações da Linguagem

A Tabela 1 apresenta as especificidades de design da linguagem **HolyPy**, definidas com o objetivo de criar uma linguagem que une a legibilidade do Python ao controle de hardware do C.

Tabela 1: Especificações Genéricas da Linguagem **HolyPy**.

Especificação
Linguagem híbrida *High-level* & *Low-level*.
Sintaxe *Python-like* com referências estruturais e tipos de C.
Sem necessidade de ponto e vírgula no final dos comandos – o fim do comando é determinado pela quebra de linha.
Delimitação de blocos e escopos obrigatoriamente por chaves `{ }`, mesclando o estilo de C com a limpeza do Python.
Nomes de variáveis e funções podem ser compostos por letras e números, mas não podem iniciar com dígito.
Declaração de variáveis pela palavra reservada `let`, com tipagem explícita opcional (ex.: `let x: i32`).
Funções declaradas com `fn` e tipo de retorno explicitado (ex.: `-> i32`).
Sistema de tipos detalhado em bits: inteiros (`i8` a `i64`), *unsigned* (`u8` a `u64`), reais (`f32`, `f64`), `char`, `bool`.
Estrutura de repetição universal sob a palavra reservada `cycle { ... }`.

Especificação
Operadores lógicos legíveis: `and`, `or`, `not`.
Entrada e saída pelas funções `speak()` e `receive()`.
Tipagem híbrida: inferida e explícita.
Modos de execução especializados por diretivas (`!kernel`, `!embedded`).
Operações de memória explícitas padronizadas com `chain(n)` (equivalente a `malloc`) e `unchain(ptr)` (equivalente a `free`).
Suporte a ponteiros para estruturas de baixo nível (ex.: `let ptr: *u32 = 0x1000`).
Capacidade de código de máquina *inline* via bloco `asm { }`.

As escolhas de design refletem um equilíbrio entre Python e C/C++. A tipagem baseada em largura de bits (`i8`–`i64`, `u8`–`u64`) elimina ambiguidades que surgem em linguagens interpretadas que dependem da arquitetura para definir o tamanho de `int` ou `long`. A ausência de ponto e vírgula obrigatório reduz complexidade sintática, enquanto as chaves obrigatórias para blocos previnem erros de indentação. O `cycle` unifica `for` e `while` em uma única construção cujo fluxo é controlado por condições e pelo comando `free`.

3.2 Tabela de Tokens e Expressões Regulares

As tabelas a seguir apresentam a definição completa dos tokens da linguagem **HolyPy**, organizados por categoria semântica, com a expressão regular correspondente.

Tabela 2: Tokens – Identificadores e Literais

Tipo	C	HolyPy	**Analisador**	Token C	Token HolyPy
Variável	`letra`	`letra`	Identificador	`T_ID`	`T_ID`
Variável	`letra+num`	`letra+num`	Identificador	`T_ID`	`T_ID`
Literal Numérico	`número`	`número`	Lit. Numérico	`T_NUM`	`T_NUM`

Tabela 3: Tokens – Operadores de Atribuição e Relacionais

Tipo	C	HolyPy	**Analisador**	Token C	Token HolyPy
Atribuição	=	=	Op. Atribuição	T_ATR	T_ATR
Relacional	==	==	Op. Relacional	T_EQ	T_EQ
Relacional	!=	!=	Op. Relacional	T_NEQ	T_NEQ
Relacional	>	>	Op. Relacional	T_GT	T_GT
Relacional	<	<	Op. Relacional	T_LT	T_LT
Relacional	>=	>=	Op. Relacional	T_GEQ	T_GEQ
Relacional	<=	<=	Op. Relacional	T_LEQ	T_LEQ

Tabela 4: Tokens – Operadores Lógicos

Tipo	C	HolyPy	**Analisador**	Token C	Token HolyPy
Lógico	&&	and	Op. Lógico	T_AND	T_AND
Lógico	||	or	Op. Lógico	T_OR	T_OR
Lógico	!	not	Op. Lógico	T_NOT	T_NOT

Os operadores lógicos do C (`&&`, `||`, `!`) são substituídos pelas palavras `and`, `or` e `not`, o que aumenta a legibilidade e elimina ambiguidades com outros usos dos símbolos `&` e `|` (operações bitwise). Do ponto de vista léxico, esses tokens são palavras reservadas e, portanto, verificados antes do padrão genérico `T_ID`.

Tabela 5: Tokens – Operadores Aritméticos

Tipo	C	HolyPy	**Analisador**	Token C	Token HolyPy
Aritmético	+	+	Op. Aritmético	T_PLUS	T_PLUS
Aritmético	-	-	Op. Aritmético	T_MINUS	T_MINUS
Aritmético	*	*	Op. Aritmético	T_MUL	T_MUL
Aritmético	/	/	Op. Aritmético	T_DIV	T_DIV
Aritmético	%	mod	Op. Aritmético	T_MOD	T_MOD

Tabela 6: Tokens – Delimitadores e Símbolos Especiais

Tipo	C	HolyPy	**Analisador**	Token C	Token HolyPy
Delimitador	{	{	Delimitador	T_LBRACE	T_LBRACE
Delimitador	}	}	Delimitador	T_RBRACE	T_RBRACE
Delimitador	(	(	Delimitador	T_LPAREN	T_LPAREN
Delimitador	)	)	Delimitador	T_RPAREN	T_RPAREN
Símbolo	:	:	Símbolo Especial	T_COLON	T_COLON
Símbolo	,	,	Símbolo Especial	T_COMMA	T_COMMA
Símbolo	->	->	Símbolo Especial	-	T_ARROW
Diretiva	#	!	Símbolo Especial	T_POUND	T_BANG
Comentário	//,/* */	#	Símbolo Especial	T_COMMENT	T_COMMENT
Delimitador	"	"	Símbolo Especial	T_STR	T_STR
Delimitador	’	’	Símbolo Especial	T_SQUOTE	T_SQUOTE

Tabela 7: Tokens – Palavras Reservadas

Categoria	C	HolyPy	**Analisador**	Token C	Token HolyPy
Fluxo	`if`	`if`	Pal. Reservada	`T_IF`	`T_IF`
Fluxo	`else`	`else`	Pal. Reservada	`T_ELSE`	`T_ELSE`
Fluxo	`for/while`	`cycle`	Pal. Reservada	`T_FOR/WHILE`	`T_CYCLE`
Controle	`break`	`free`	Pal. Reservada	`T_BREAK`	`T_FREE`
Tipo Nulo	`void`	`u0`	Pal. Reservada	`T_VOID`	`T_U0`
Função	-	`fn`	Pal. Reservada	-	`T_FN`
Retorno	`return`	`return`	Pal. Reservada	`T_RETURN`	`T_RETURN`
E/S	`printf`	`speak`	Pal. Reservada	`T_PRINTF`	`T_SPEAK`
E/S	`scanf`	`receive`	Pal. Reservada	`T_SCANF`	`T_RECEIVE`
Importação	`include`	`import`	Pal. Reservada	`T_INCLUDE`	`T_IMPORT`
Memória	`malloc`	`chain`	Pal. Reservada	`T_MALLOC`	`T_CHAIN`
Memória	`free`	`unchain`	Pal. Reservada	`T_FREE`	`T_UNCHAIN`
Baixo Nível	`asm`	`asm`	Pal. Reservada	`T_ASM`	`T_ASM`
Booleano	`true`	`true`	Pal. Reservada	`T_TRUE`	`T_TRUE`
Booleano	`false`	`false`	Pal. Reservada	`T_FALSE`	`T_FALSE`

Tabela 8: Expressões Regulares dos Tokens de **HolyPy**.

Token	**Expressão Regular**	**Exemplo**
`T_ID`	`[A-Za-z_][A-Za-z0-9_]*`	variavel1, _soma

`T_NUM`	`[+-]?(?:0[xX][0-9A-Fa-f]+|`
	`(?:[0-9]+(?:\.[0-9]*)?|`
	`string[0-9]+)`
	`(?:[eE][+-]?[0-9]+)?)|`
		42, -3.14, 0xFF

`T_CHAR_LIT`	`'(?:\\.|[^'\\])'`	'a', '\n'

Token	Expressão Regular	Exemplo
T_STR	"(?:\\.|[^"\\])*"	"Ola", "linha\n"
T_COMMENT	#.*	# comentário
T_ATR / T_EQ	= | ==	=, ==
T_NEQ	!=	!=
T_GEQ, T_GT	>= | >	>=, >
T_LEQ, T_LT	<= | <	<=, <
T_PLUS, T_MINUS	\+ | -	+, -
T_MUL, T_DIV	* | /	*, /
T_MOD	\bmod\b	mod
T_AND, T_OR, T_NOT	\band\b| \bor\b| \bnot\b	and, or, not
T_LPAREN, T_RPAREN	\(| \)	(,)
T_LBRACE, T_RBRACE	\{ | \}	{, }
T_COLON, T_COMMA	: | ,	:, ,
T_ARROW, T_BANG	-> | !	->, !
T_INT	i(8|16|32|64)	i32, i8
T_UINT	u(8|16|32|64)	u16, u64
T_FLOAT	f(32|64)	f32, f64
T_CHAR, T_BOOL, T_U0	char | bool | u0	char, u0

Token	Expressão Regular	Exemplo
Pal. Res. (fluxo)	\blet\b| \bfn\b| \bif\b| ...	let, fn, if, cycle
Pal. Res. (I/O+mem.)	\bspeak\b| \breceive\b| ...	speak, chain
Pal. Res. (bool)	\btrue\b| \bfalse\b	true, false

Detalhamento das Expressões Regulares Complexas

T_NUM. A ER aceita um sinal opcional ([+-]?), seguido de dois ramos alternativos: (1) hexadecimais iniciados com 0x ou 0X seguidos de dígitos hexadecimais ([0-9A-Fa-f]+); (2) números decimais com parte inteira obrigatória ou ponto inicial (\.[0-9]+), ambos com expoente opcional ([eE][+-]?[0-9]+). Isso cobre inteiros, decimais, hexadecimais e notação científica em uma única regra.

T_CHAR_LIT. Abre e fecha com aspas simples. Aceita exatamente um token que é: (a) uma sequência de escape (\\., barra seguida de qualquer caractere, cobrindo '\n', '\\', etc.) ou (b) qualquer caractere que não seja aspas simples nem barra ([^'\\]). A ausência de quantificador * ou + restringe a exatamente um caractere.

T_STR. Idêntica à lógica de T_CHAR_LIT, mas usa aspas duplas e permite zero ou mais caracteres (*), cobrindo strings vazias. O padrão [^"\\] impede que uma aspa dupla interna feche a string prematuramente.

T_ID vs. palavras reservadas. A resolução é feita com estratégia de mapeamento pós-captura: o lexer casa o padrão [A-Za-z_][A-Za-z0-9_]* e, em seguida, verifica se o lexema pertence à tabela de palavras reservadas, reclassificando o token quando necessário. Isso centraliza a lógica em uma única ER e mantém a tabela de palavras-chave de fácil entedimento.

3.3 Regras de Produção

A Tabela 9 apresenta as regras gramaticais que definem a sintaxe de **HolyPy**. Cada produção nomeia uma construção da linguagem e especifica a sequência de tokens esperada, com os operadores `?` (zero ou uma ocorrência) e `+` (uma ou mais) aplicados sobre subsequências entre parênteses.

Tabela 9: Regras de Produção da Gramática de **HolyPy**.

Produção	Regra
Condicional	T_IF condicao T_LBRACE comandos+ T_RBRACE (T_ELSE T_LBRACE comandos+ T_RBRACE)?
Repetição	T_CYCLE T_LBRACE comandos+ T_RBRACE (T_FREE)?
DecVariável	T_LET T_ID (T_COLON tipos)? (T_ATR (expr)?)?
Atribuição	T_ID T_ATR (caracteres | opArit | opLog) T_SEMI?
DecFunção	T_FN T_ID T_LPAREN params T_RPAREN T_ARROW tipos T_LBRACE comandos+ T_RETURN expr T_RBRACE
CallFunção	T_ID T_LPAREN params T_RPAREN
Speak	T_SPEAK T_LPAREN (expr | T_STR | T_COMMA)* T_RPAREN
Receive	T_RECEIVE T_LPAREN T_ID T_RPAREN
Comentário	T_COMMENT
Chain	T_CHAIN T_LPAREN (T_NUM | T_ID) T_RPAREN
Unchain	T_UNCHAIN T_LPAREN T_ID T_RPAREN
Bloco ASM	T_ASM T_LBRACE (T_STR)+ T_RBRACE
Indexação	T_ID T_LBRACKET (T_NUM | T_ID | opArit) T_RBRACKET
Acesso Membro	T_ID T_DOT T_ID
DecPonteiro	T_LET T_ID T_COLON T_STAR tipos (T_ATR (T_NUM | T_ID))?

Produção	Regra
Lit. Booleano	T_TRUE | T_FALSE

A Tabela 10 detalha a finalidade de cada regra de produção e apresenta um exemplo de uso concreto em código **HolyPy**.

Tabela 10: Explicação das Regras de Produção de **HolyPy**.

Nome	Finalidade	Exemplo de Uso

Programa	Unidade sintática raiz: sequência de diretivas, declarações e funções.	`!kernel`
		`import stdio`
		`fn main() -> i32 {`
		`return 0 }`

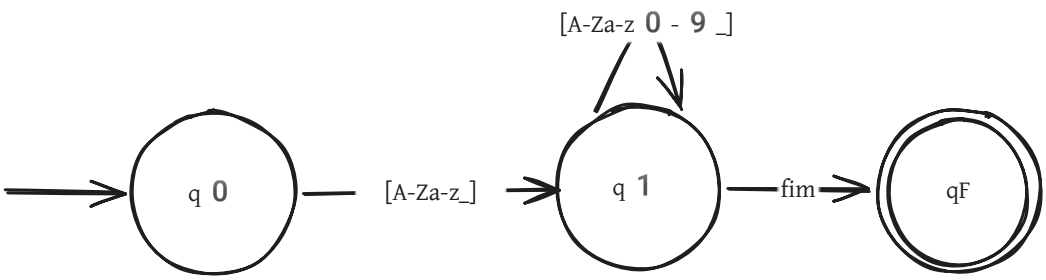
Diretiva de Modo	Ativa modo especial de compilação via diretiva (`!`).	`!embedded`
Importação	Importa módulos/arquivos para o escopo do programa.	`import math`
Declaração de Função	Declara função com nome, parâmetros, tipo de retorno e bloco.	`fn soma(a: i32, b: i32) -> i32 { return a + b }`
Parâmetros de Função	Define lista de parâmetros tipados (ou vazia) em assinatura/chamada.	`fn inc(x: i32) -> i32 { return x+1 }`
Tipo Estrutural	Restringe tipos válidos como tokens próprios, não `T_ID`.	`let x: i32 = 10` `fn f() -> u0 { }`
Bloco	Delimita escopo de comandos com `{ }`.	`if x > 0 { speak("ok") }`
Declaração de Variável	Declara variável com tipagem e inicialização opcionais.	`let a` `let b: f64 = 3.14`
Atribuição	Atribui valor a identificador já declarado.	`total = total + 1`
Chamada de Função	Invoca rotina com argumentos posicionais.	`soma(2, 3)`
Retorno	Finaliza função retornando expressão compatível com o tipo de retorno.	`return resultado`

Nome	Finalidade	Exemplo de Uso
Condicional	Executa bloco condicionalmente, com ramo alternativo opcional.	`if n >= 0 { speak("pos") } else { speak("neg") }`
Repetição	Executa bloco em laço; parada por condição interna e/ou `free`.	`cycle { if i >= 10 { free } i = i + 1 }`
Interrupção de Laço	Interrompe laço `cycle` imediatamente.	`free`
Saída	Emite dados para saída padrão.	`speak("Soma:", total)`
Entrada	Lê valor de entrada para variável alvo.	`receive(x)`
Expressão Relacional	Compara operandos com `=`, `==`, `!=`, `>`, `<`, `>=`, `<=`.	`if a <= b { ... }`
Expressão Lógica	Combina condições com `and`, `or`, `not`.	`if not erro and ativo { ... }`
Expressão Aritmética	Realiza operações `+`, `-`, `*`, `/`, `mod`.	`media = soma / n`
Literal Numérico	Aceita inteiro/float com sinal, hexadecimal e expoente.	`-42 +2.5e3 0xFF`
Literal de Caractere	Representa um caractere simples ou sequência de escape.	`'a' '\n'`
Literal de String	Representa sequência textual com escapes válidos.	`"linha\n" "ok"`
Identificador	Nome de variável/função iniciado por letra ou `_`.	`_tmp1 contador`
Pal. Reservada vs. T_ID	Garante prioridade de keywords/tipos sobre `T_ID`.	`if` é keyword, não ID
Operadores Compostos	Garante casamento de `->`, `!=`, `>=`, `<=`, `==` antes dos simples.	`fn f() -> i32 { return 0 }`
Indexação de Array	Acesso a elementos por posição com `[ ]`.	`v[0]`

Nome	Finalidade	Exemplo de Uso
Acesso a Membro	Seleciona campo de estrutura com `.`	`obj.campo`
Separadores	Separação explícita com `;` ou quebra de linha.	`let x: i32 = 1; let y: i32 = 2`
Comentário de Linha	Ignora texto após `#` até fim da linha.	# comentário

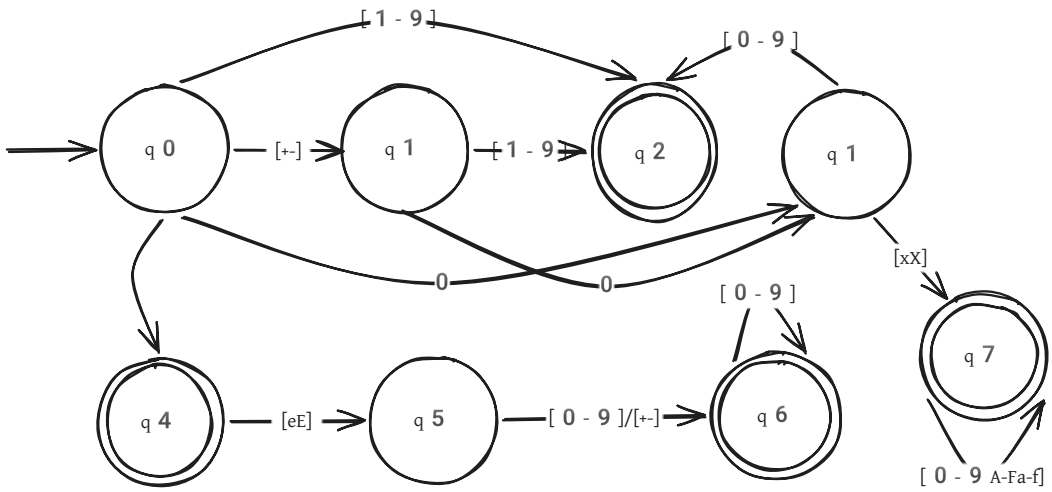
3.4 Autômatos Finitos

Os diagramas a seguir apresentam os AFDs construídos para os principais tokens da linguagem. Cada figura é referenciada pelo nome do token e exibe a expressão regular que o originou na legenda. Os diagramas foram gerados com o Excalidraw e exportados em formato SVG.



ER: `[A-Za-z_][A-Za-z0-9_]*`

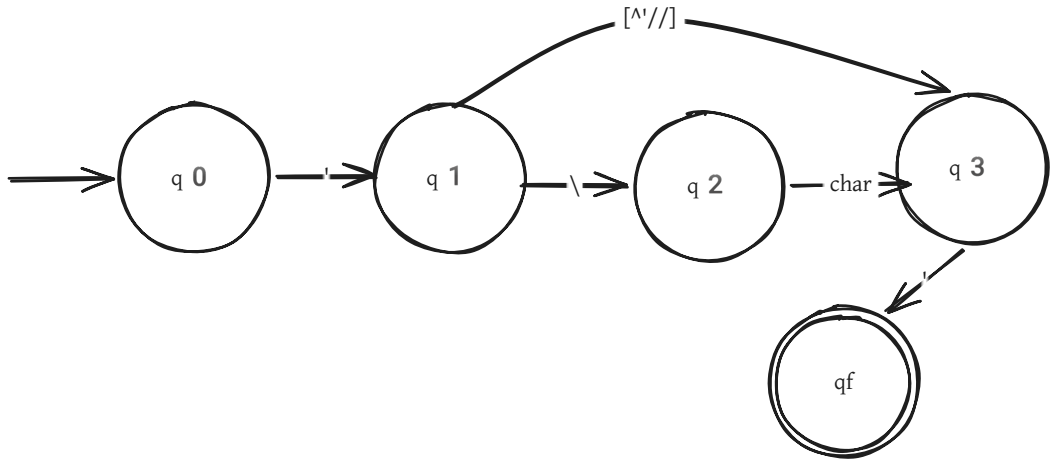
Figura 2: AFD do token `T_ID` (Identificadores)



ER:

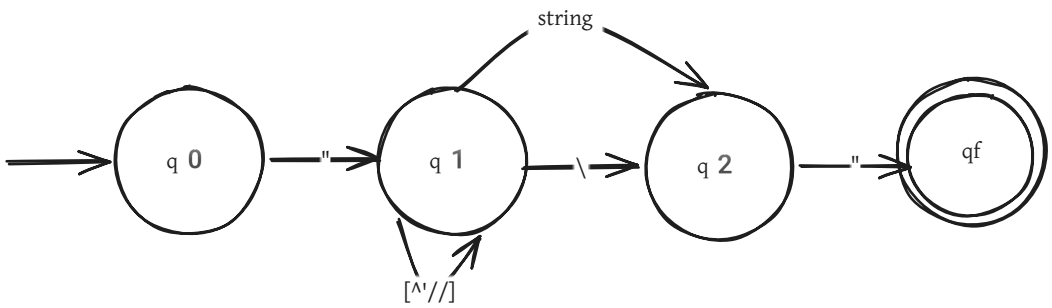
`[+-]?(?:0[xX][0-9A-Fa-f]+|(?:[0-9]+(?:\.[0-9]*)?|\.[0-9]+)(?:[eE][+-]?[0-9]+)?)|`

Figura 3: AFD do token `T_NUM` (Literais Numéricos)



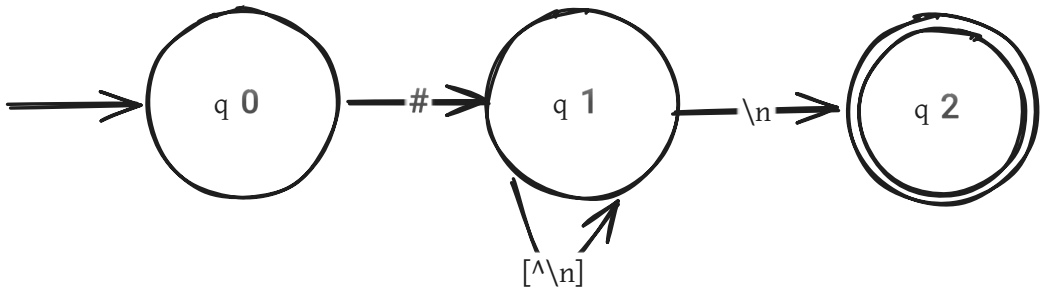
ER: `'(?:\.|[\^{}'\])'`

Figura 4: AFD do token `T_CHAR_LIT` (Literal de Caractere)



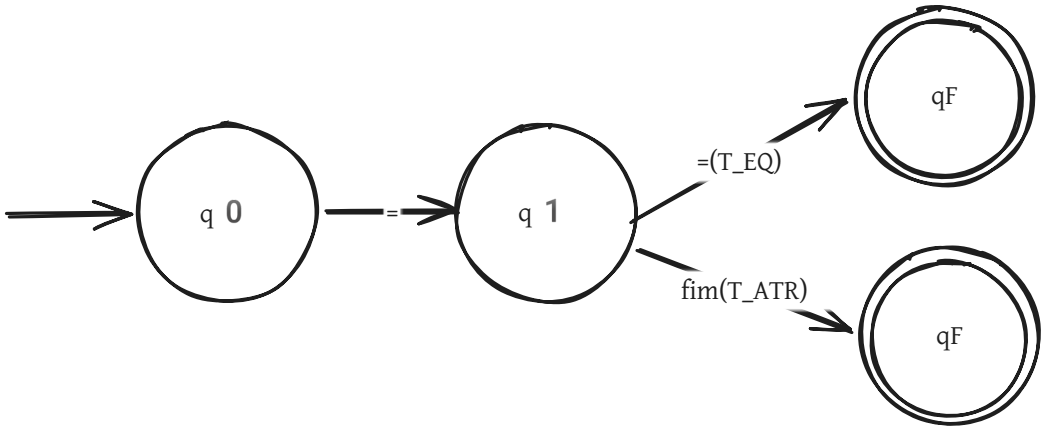
ER: `"(?:\.|[\^{}"\])*"`

Figura 5: AFD do token `T_STR` (Literal de String)



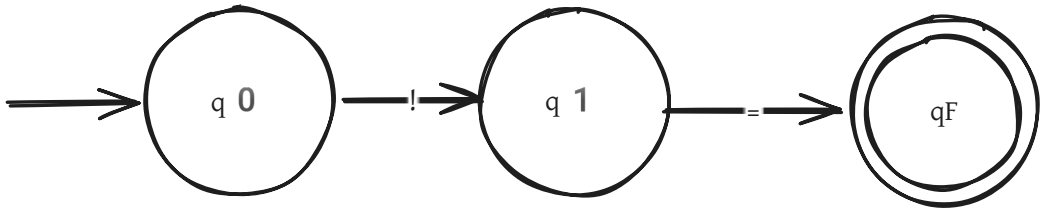
ER: `\#.*`

Figura 6: AFD do token `T_COMMENT` (Comentário de Linha)



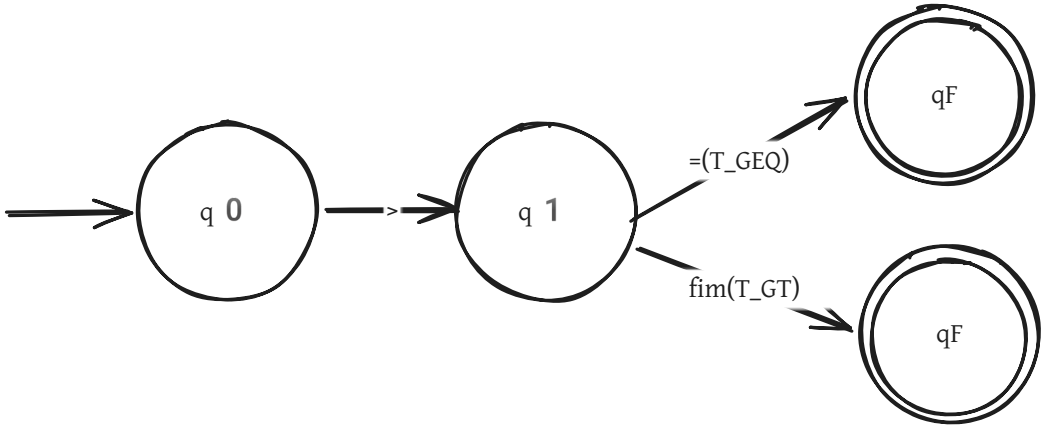
ER: `=|==`

Figura 7: AFD dos tokens `T_ATR` e `T_EQ` (= e ==)



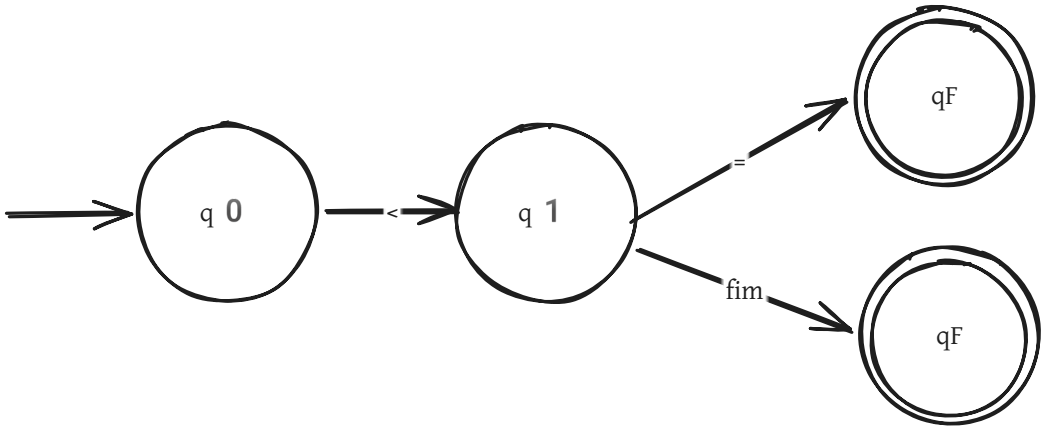
ER: `!=`

Figura 8: AFD do token `T_NEQ` (`!=`)



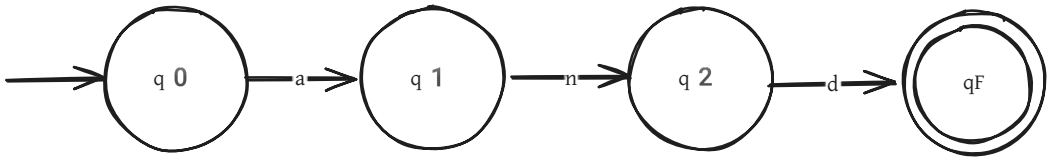
ER: `>=|>`

Figura 9: AFD dos tokens `T_GT` e `T_GEQ` (`>` e `>=`)



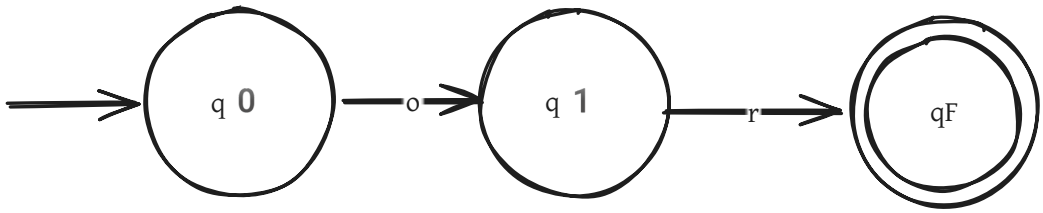
ER: `<=|<`

Figura 10: AFD dos tokens `T_LT` e `T_LEQ` (`<` e `<=`)



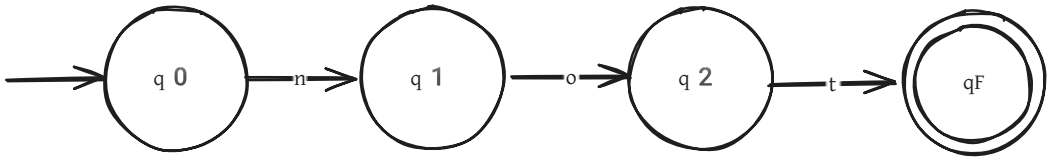
ER: `\band \b`

Figura 11: AFD do token `T_AND` (`and`)



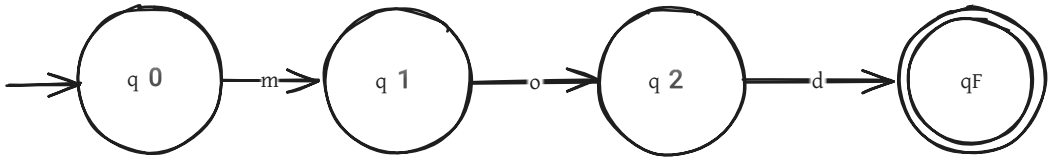
ER: `\bor \b`

Figura 12: AFD do token `T_OR` (`or`)



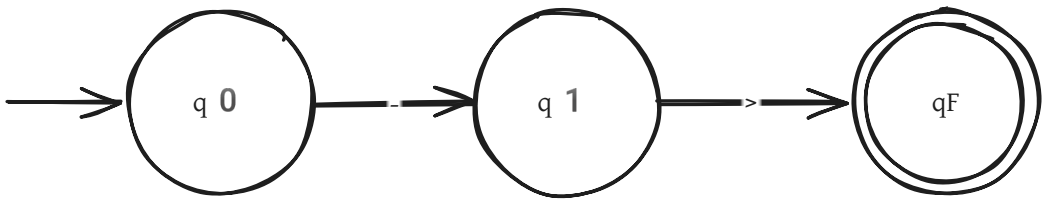
ER: `\bnot \b`

Figura 13: AFD do token `T_NOT` (`not`)



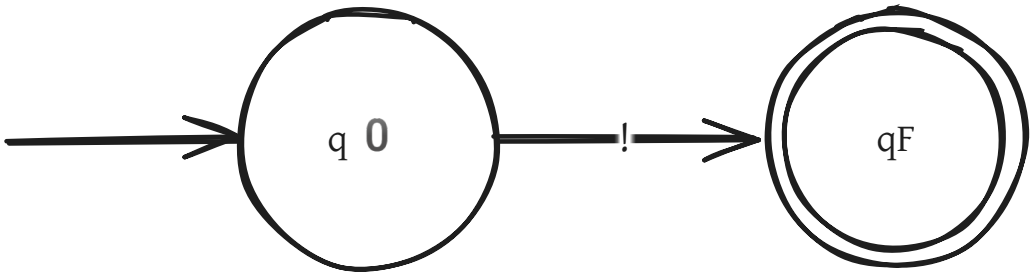
ER: `\bmod \b`

Figura 14: AFD do token `T_MOD` (`mod`)



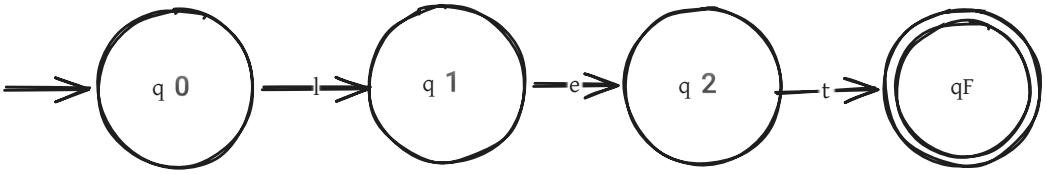
ER: `->`

Figura 15: AFD do token `T_ARROW` (`->`)



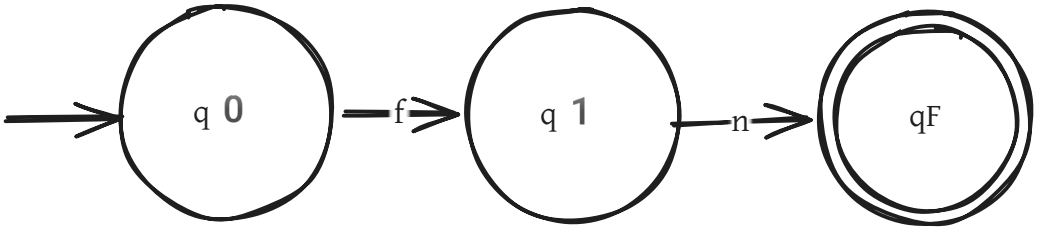
ER: `!`

Figura 16: AFD do token `T_BANG` (`!`)



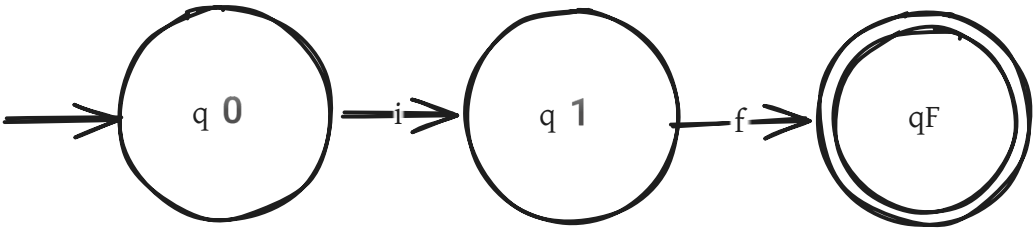
ER: `\blet \b`

Figura 17: AFD do token `T_LET` (`let`)



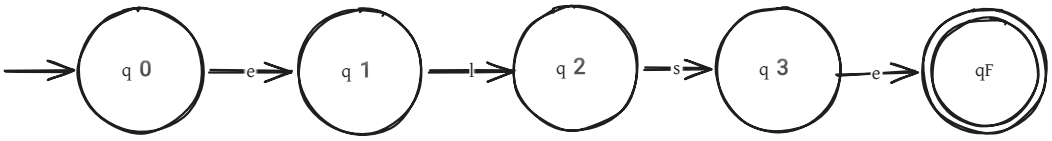
ER: `\bfn \b`

Figura 18: AFD do token `T_FN` (`fn`)



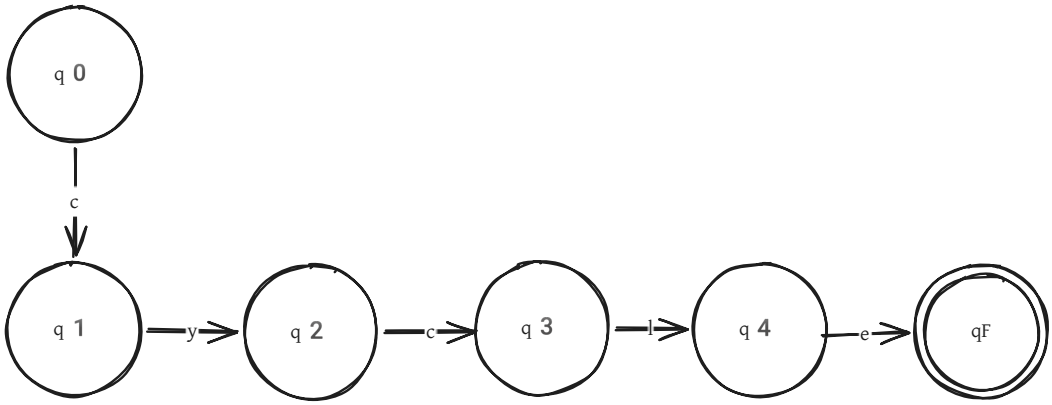
ER: `\bif \b`

Figura 19: AFD do token `T_IF` (`if`)



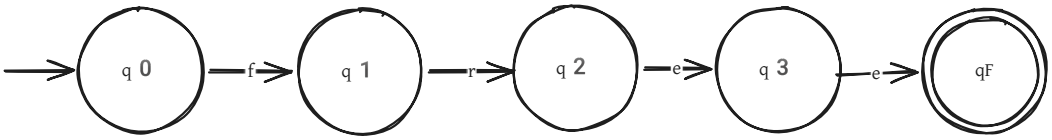
ER: `\belse \b`

Figura 20: AFD do token `T_ELSE` (`else`)



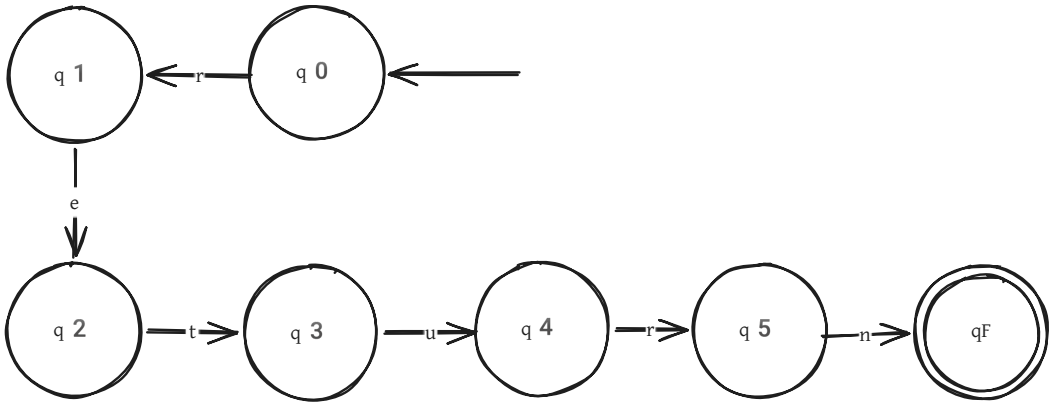
ER: `\bcycle \b`

Figura 21: AFD do token `T_CYCLE` (`cycle`)



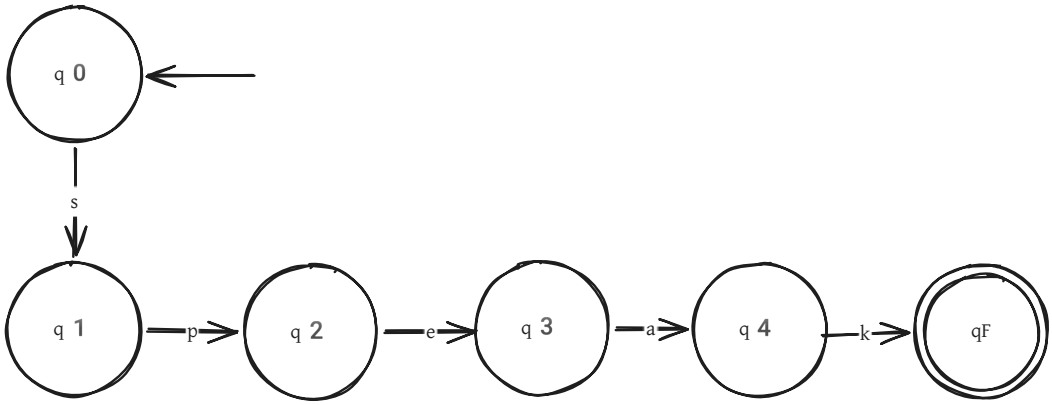
ER: `\bfree \b`

Figura 22: AFD do token `T_FREE` (`free`) — interrupção de laço



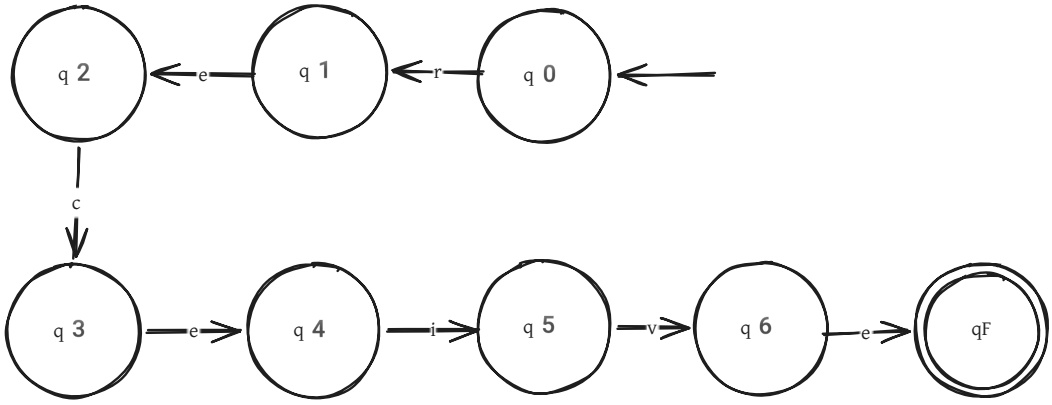
ER: `\breturn \b`

Figura 23: AFD do token `T_RETURN` (`return`)



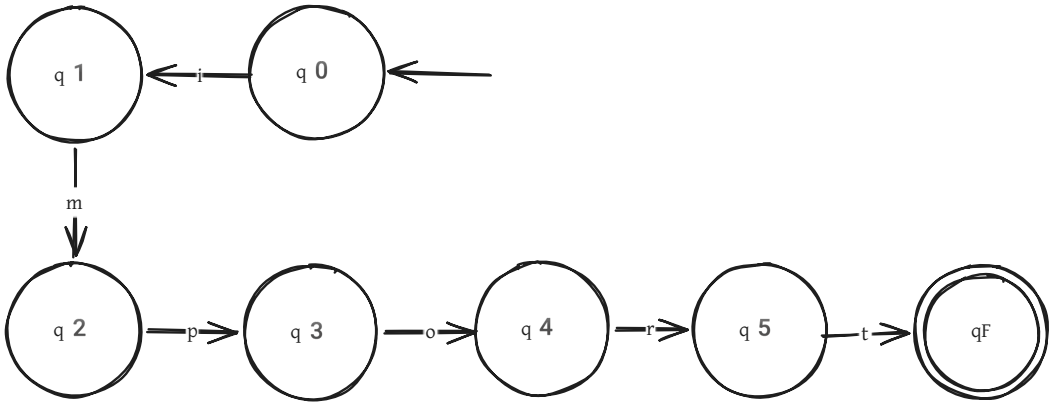
ER: `\bspeak \b`

Figura 24: AFD do token `T_SPEAK` (`speak`)



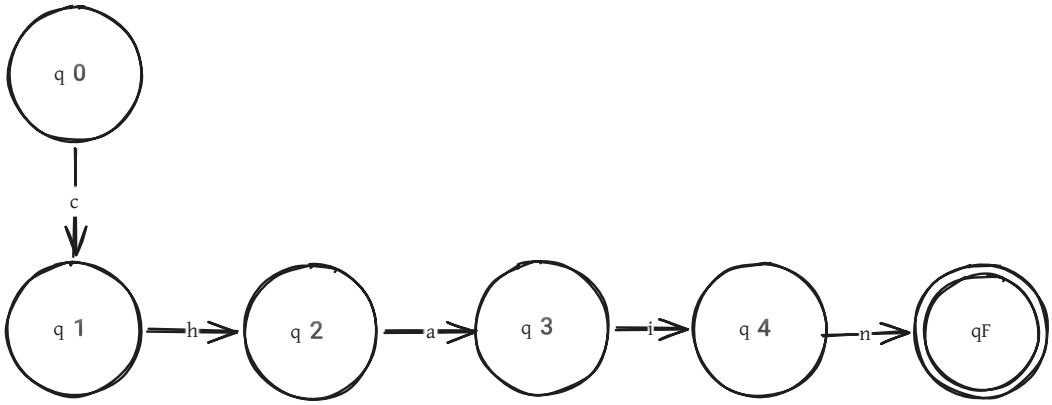
ER: `\breceive \b`

Figura 25: AFD do token `T_RECEIVE` (`receive`)



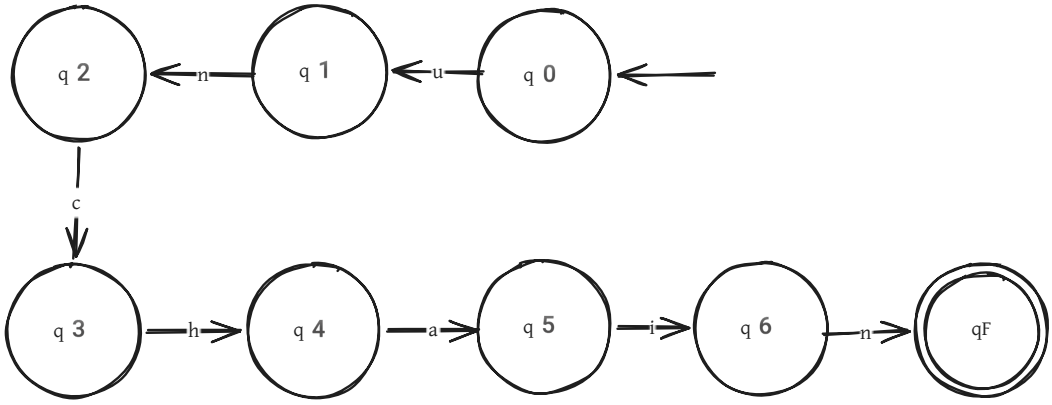
ER: `\bimport \b`

Figura 26: AFD do token `T_IMPORT` (`import`)



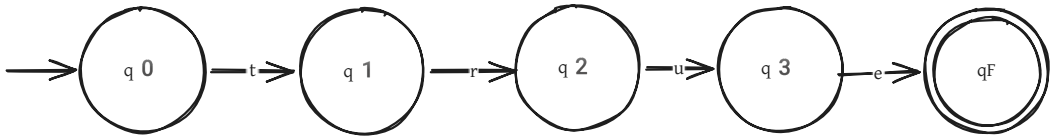
ER: `\bchain \b`

Figura 27: AFD do token `T_CHAIN` (`chain`)



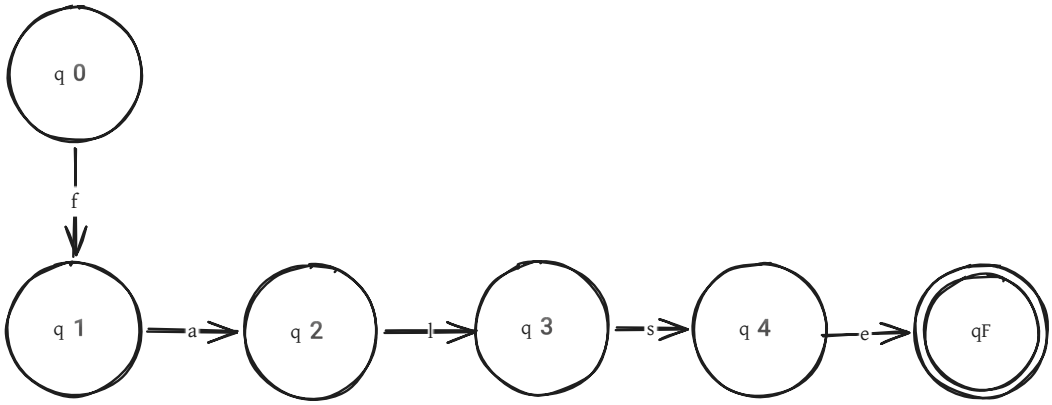
ER: `\bunchain \b`

Figura 28: AFD do token `T_UNCHAIN` (`unchain`)



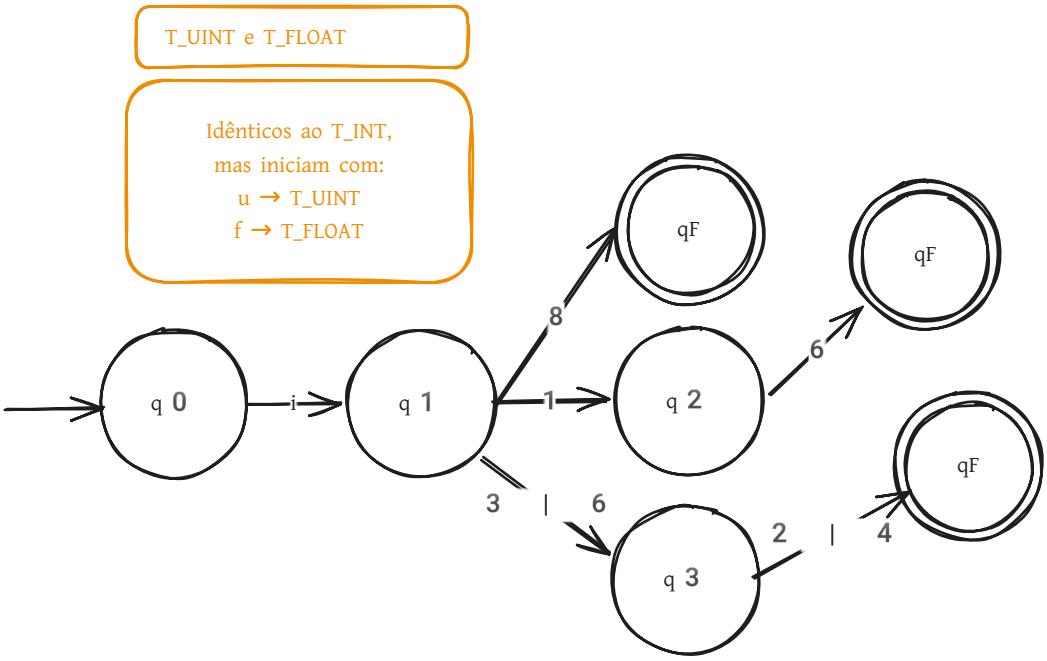
ER: `\btrue \b`

Figura 29: AFD do token `T_TRUE` (`true`)



ER: `\bfalse \b`

Figura 30: AFD do token `T_FALSE` (`false`)



ER: `i(8|16|32|64) | u(8|16|32|64) | f(32|64) | char | bool | u0`

Figura 31: AFD dos Tipos Estruturais (`T_INT`, `T_UINT`, `T_FLOAT`)

4 Conclusão

O desenvolvimento dessa primeira parte da linguagem **HolyPy** permitiu percorrer, de forma prática e estruturada, as fases iniciais do processo de compilação. A definição do vocabulário evidenciou as decisões de design que impactam diretamente a complexidade

das expressões regulares: a escolha de palavras legíveis para operadores lógicos (`and`, `or`, `not`) simplificou a distinção entre operadores e identificadores; a tipagem por bits (`i8`–`i64`) exigiu uma ER composta com alternância, mas tirou ambiguidades de tamanho dependente de plataforma.

Cada estado representa uma decisão de leitura do próximo caractere, e a minimização dos autômatos resultou em máquinas eficientes, sem estados redundantes.

As regras de produção definiram a gramática livre de contexto que guia o *parser* de descida recursiva. A escolha de uma gramática sem ambiguidades para as expressões aritméticas e lógicas é um ponto central para a funcionalidade correta do analisador sintático.

4.1 Disponibilização e documentação do projeto

O projeto **HolyPy** está disponível publicamente em repositório no perfil do autor, no endereço `https://github.com/arraesgustavo/HolyPy/tree/main`. Nesse repositório estão presentes materiais complementares que documentam seu desenvolvimento.

Os textos explicativos do projeto foram organizados no diretório `docs/`, feitos em arquivos `.md`, formato usado por ser o padrão de documentação do GitHub e por favorecer leitura direta no ambiente do repositório. Os autômatos construídos ao longo do trabalho estão disponíveis em `docs/automata/`, enquanto as tabelas utilizadas na documentação foram originalmente elaboradas no LibreOffice Calc, armazenadas em `docs/sheets/` e posteriormente convertidas para LaTeX com o uso de ferramentas apropriadas.

Além disso, o repositório preserva outros elementos importantes do processo de desenvolvimento, como registros do histórico de implementação, detalhes adicionais sobre as decisões de projeto, o enunciado do trabalho, as referências utilizadas e demais documentos produzidos durante a disciplina. Este *.pdf*, por sua vez, foi escrita em LaTeX, com apoio da ferramenta Prism, usada como ambiente de edição e organização do documento.

Referências

[1] AHO, A. V.; SETHI, R.; ULLMAN, J. D. *Compiladores: Princípios, Técnicas e Ferramentas*. Editora Livros Técnicos e Científicos, 1995.

[2] PRICE, Ana M. de A.; TOSCANI, Simão S. *Implementação de Linguagens de Programação: Compiladores*. Série Livros Didáticos, n. 9. Editora Sagra Luzzatto, 2000.

[3] LOUDEN, Kenneth C. *Compiladores: Princípios e Práticas*. Pioneira Thomson Learning, 2004.

[4] SERAPHIM, Thatyana de Faria Piola. *Slides da disciplina de Compiladores*. Material didático da disciplina, Universidade Federal de Itajubá (UNIFEI), 2026.

[5] SERAPHIM, Thatyana de Faria Piola. *Slides da disciplina de Linguagens Formais*. Material didático da disciplina, Universidade Federal de Itajubá (UNIFEI), 2025.

[6] DAVIS, T. A. *TempleOS – Holy C Language Reference*. Disponível em: https://templeos.org. Acesso em: 2026.

[7] MATSAKIS, N. D.; KLOCK II, F. S. The Rust language. *ACM SIGAda Ada Letters*, v. 34, n. 3, p. 103–104, 2014.